OpenCV: 10 Practicals

Complete Python programs formatted for copying into VS Code

Before running: Install the required packages with `pip install opencv-python numpy matplotlib`. Place the image files named in each program in the same folder as your Python script. Webcam practicals require a working camera.

Note: Q1–Q6 are organized from the practical topics and code details shared in the conversation. If your teacher's original version uses different image filenames, change the filename in `cv2.imread()`.

Contents

Q1. Basic Image Operations (7 examples)

Q2. Colour Space Conversion

Q3. Brightness, Contrast and Blurring

Q4. Image Histograms (7 examples)

Q5. Edge Detection: Sobel and Canny

Q6. Harris Corner Detection

Q7. ORB Feature Detection and Matching

Q8. Image Segmentation: Binary, Adaptive and K-Means

Q9. Real-Time Face Detection

Q10. Real-Time Motion Detection

Q1. Basic Image Operations

Each example is a separate runnable program. Use one example at a time in VS Code.

1.1 Convert image to grayscale

```
import cv2

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow("Original Image", image)
cv2.imshow("Grayscale Image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.2 Resize image

```
import cv2

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

resized = cv2.resize(image, (500, 400))
cv2.imshow("Resized Image", resized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.3 Crop image

```
import cv2

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

cropped = image[50:250, 100:300]
cv2.imshow("Cropped Image", cropped)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.4 Rotate image 90 degrees clockwise

```
import cv2

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

rotated = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)
cv2.imshow("Rotated Image", rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.5 Flip image

```
import cv2

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

flipped = cv2.flip(image, 1)  # 1 = horizontal flip
cv2.imshow("Flipped Image", flipped)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.6 Access pixel values

```
import cv2

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

print("Pixel value at (100, 100):", image[100, 100])
cv2.imshow("Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.7 Change a pixel

```
import cv2
```

```
image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

image[100, 100] = [0, 0, 255]  # BGR: red
cv2.imshow("Modified Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Q2. Colour Space Conversion

Grayscale and HSV conversion

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

print("BGR pixel at (100, 100):", image[100, 100])
print("Grayscale pixel at (100, 100):", gray[100, 100])
print("HSV pixel at (100, 100):", hsv[100, 100])

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Q3. Brightness, Contrast and Blurring

3.1 Adjust brightness and contrast

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

adjusted = cv2.convertScaleAbs(image, alpha=1.5, beta=50)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(adjusted, cv2.COLOR_BGR2RGB))
plt.title("Brightness and Contrast Adjusted")
plt.axis("off")

plt.tight_layout()
plt.show()
```

3.2 Gaussian blur and median blur

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

gaussian = cv2.GaussianBlur(image, (5, 5), 0)
median = cv2.medianBlur(image, 5)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(gaussian, cv2.COLOR_BGR2RGB))
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(median, cv2.COLOR_BGR2RGB))
plt.title("Median Blur")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Q4. Image Histograms

The following seven examples use nature.jpg. Run each example separately.

4.1 Grayscale histogram

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])

plt.plot(hist)
plt.title("Grayscale Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.show()
```

4.2 BGR colour histogram

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

for channel, color in enumerate(("b", "g", "r")):
    hist = cv2.calcHist([image], [channel], None, [256], [0, 256])
    plt.plot(hist, color=color, label=color.upper())

plt.title("BGR Colour Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.legend()
plt.show()
```

4.3 Two-dimensional Hue-Saturation histogram

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0, 1], None, [180, 256], [0, 180, 0, 256])
hist = cv2.normalize(hist, None, 0, 255, cv2.NORM_MINMAX)

plt.imshow(hist, interpolation="nearest", aspect="auto")
plt.title("2D Hue-Saturation Histogram")
plt.xlabel("Saturation")
plt.ylabel("Hue")
plt.show()
```

4.4 Normalized histogram

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
hist = cv2.normalize(hist, None, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)

plt.plot(hist)
plt.title("Normalized Grayscale Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Normalized Frequency")
plt.xlim([0, 256])
plt.show()
```

4.5 Cumulative histogram

```
import cv2
import matplotlib.pyplot as plt
```

```
import numpy as np

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256]).flatten()
cumulative = np.cumsum(hist)

plt.plot(cumulative)
plt.title("Cumulative Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Cumulative Frequency")
plt.xlim([0, 256])
plt.show()
```

4.6 Histogram equalization

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
equalized = cv2.equalizeHist(gray)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized")
plt.axis("off")

plt.tight_layout()
plt.show()

cv2.imshow("Equalized Histogram", equalized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

4.7 Histogram with a mask / region of interest (ROI)

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

image = cv2.imread("nature.jpg")
if image is None:
    raise FileNotFoundError("Could not load nature.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
mask = np.zeros(gray.shape, dtype=np.uint8)
height, width = gray.shape
cv2.rectangle(mask, (width // 4, height // 4),
              (3 * width // 4, 3 * height // 4), 255, -1)

hist = cv2.calcHist([gray], [0], mask, [256], [0, 256])

plt.plot(hist)
plt.title("Histogram of Masked ROI")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.show()

plt.figure(figsize=(5, 4))
plt.imshow(mask, cmap="gray")
plt.title("ROI Mask")
plt.axis("off")
plt.show()
```

Q5. Edge Detection: Sobel and Canny

5.1 Sobel edge detection

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("building.jpg")
if image is None:
    raise FileNotFoundError("Could not load building.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)
combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(combined, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")

plt.tight_layout()
plt.show()
```

5.2 Canny edge detection

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("road.jpg")
if image is None:
    raise FileNotFoundError("Could not load road.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blurred, 100, 200)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Q6. Harris Corner Detection

Harris corner detector

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("building.jpg")
if image is None:
    raise FileNotFoundError("Could not load building.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

corners = cv2.cornerHarris(gray, 2, 3, 0.04)
corners = cv2.dilate(corners, None)

result = image.copy()
result[corners > 0.01 * corners.max()] = [0, 0, 255]

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Harris Corner Detection")
plt.axis("off")
plt.show()
```

Q7. ORB Feature Detection and Matching

ORB matching between Building.jpg and cat.jpg

```
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread("Building.jpg")
img2 = cv2.imread("cat.jpg")

if img1 is None or img2 is None:
    raise FileNotFoundError("Could not load Building.jpg or cat.jpg")

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

orb = cv2.ORB_create(nfeatures=1000)
keypoints1, descriptors1 = orb.detectAndCompute(gray1, None)
keypoints2, descriptors2 = orb.detectAndCompute(gray2, None)

print("Keypoints in Image 1:", len(keypoints1))
print("Keypoints in Image 2:", len(keypoints2))

if descriptors1 is None or descriptors2 is None:
    raise ValueError("Could not find descriptors in one or both images.")

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(descriptors1, descriptors2)
matches = sorted(matches, key=lambda x: x.distance)
best_matches = matches[:50]

result = cv2.drawMatches(
    img1, keypoints1, img2, keypoints2, best_matches, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 8))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("ORB Feature Detection and Matching")
plt.axis("off")
plt.show()

print("Total matches:", len(matches))
print("Best matches displayed:", len(best_matches))
```

Q8. Image Segmentation

This practical contains three separate programs. Run each one separately.

8.1 Binary thresholding

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("cat.jpg")
if image is None:
    raise FileNotFoundError("Could not load cat.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(binary, cmap="gray")
plt.title("Binary Threshold")
plt.axis("off")

plt.tight_layout()
plt.show()
```

8.2 Adaptive thresholding

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("document.jpg")
if image is None:
    raise FileNotFoundError("Could not load document.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
adaptive = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(adaptive, cmap="gray")
plt.title("Adaptive Threshold")
plt.axis("off")

plt.tight_layout()
plt.show()
```

8.3 K-Means clustering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")
if image is None:
    raise FileNotFoundError("Could not load image.jpg")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
pixels = image_rgb.reshape((-1, 3)).astype(np.float32)

K = 3
criteria = (
    cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)

_, labels, centers = cv2.kmeans(
    pixels, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS
)

centers = np.uint8(centers)
segmented_data = centers[labels.flatten()]
segmented_image = segmented_data.reshape(image_rgb.shape)
```

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(segmented_image)
plt.title("K-Means Segmentation")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Q9. Real-Time Face Detection

Haar Cascade face detection using webcam

```
import cv2

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

if face_cascade.empty():
    raise RuntimeError("Could not load Haar Cascade classifier.")

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open webcam.")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(
        gray, scaleFactor=1.1, minNeighbors=5
    )

    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    cv2.imshow("Face Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Q10. Real-Time Motion Detection

Background subtraction using MOG2

```
import cv2

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open webcam.")

background_subtractor = cv2.createBackgroundSubtractorMOG2()

while True:
    ret, frame = cap.read()
    if not ret:
        break

    foreground_mask = background_subtractor.apply(frame)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    foreground_mask = cv2.morphologyEx(
        foreground_mask, cv2.MORPH_OPEN, kernel
    )

    contours, _ = cv2.findContours(
        foreground_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    for contour in contours:
        if cv2.contourArea(contour) < 500:
            continue

        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(
            frame, "Motion Detected", (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2
        )

    cv2.imshow("Motion Detection", frame)
    cv2.imshow("Foreground Mask", foreground_mask)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```